TRƯỜNG ĐẠI HỌC ĐÀ LẠT
KHOA CÔNG NGHỆ THÔNG TIN

BÀI THỰC HÀNH 1

TỔNG QUAN & THIẾT LẬP MÔI TRƯỜNG PHÁT TRIỂN

Môn học: Các công nghệ mới trong phát triển phần mềm
Thời lượng: 4 tiết (180 phút)

Đà Lạt, 2026

MỤC LỤC

Mục tiêu bài thực hành 1

Yêu cầu đầu vào 1

Quy ước ký hiệu 1

Phần 1: Cài đặt môi trường phát triển 2

- **1.1. Cài đặt Node.js** 2
- **1.2. Cài đặt Visual Studio Code** 4
- **1.3. Cài đặt Git** 5
- **Bài tập tự làm — Phần 1** 8

Phần 2: GitHub Student Developer Pack & GitHub Copilot 8

- **2.1. Đăng ký GitHub Student Developer Pack** 9
- **2.2. Kích hoạt GitHub Copilot** 9
- **2.3. Trải nghiệm GitHub Copilot trong VS Code** 10
- **Bài tập tự làm — Phần 2** 11

Phần 3: Trải nghiệm Copilot CLI & Gemini CLI 11

- **3.1. GitHub Copilot CLI** 11
- **3.2. Gemini CLI** 14
- **3.3. So sánh Copilot CLI và Gemini CLI** 17
- **Bài tập tự làm — Phần 3** 18

Phần 4: Thực hành Git workflow cơ bản 19

- **4.1. Tạo repository trên GitHub** 19
- **4.2. Clone repository về máy** 19
- **4.3. Quy trình làm việc cơ bản** 20
- **4.4. Conventional Commits** 21
- **4.5. Branching (phân nhánh)** 22
- **4.6. Giải quyết conflict** 23
- **4.7. File .gitignore** 24
- **4.8. Xem lịch sử commit** 25
- **Bài tập tự làm — Phần 4** 25

Phần 5: Khởi tạo project Next.JS đầu tiên 25

- **5.1. Khởi tạo project** 26
- **5.2. Cấu trúc thư mục** 27
- **5.3. Chạy development server** 27
- **5.4. Chỉnh sửa trang chủ** 28
- **5.5. Chỉnh sửa layout** 29

5.6. Commit và push lên GitHub 29
Bài tập tự làm — Phần 5 30
Phần 6: Tổng kết & Yêu cầu nộp bài 30
Checklist hoàn thành 30
Yêu cầu nộp bài 31
Chuẩn bị cho buổi sau 31
Tài liệu tham khảo 31

Mục tiêu bài thực hành

Sau khi hoàn thành bài thực hành này, sinh viên có khả năng:

1. Cài đặt và cấu hình thành công bộ công cụ phát triển: Node.js, Visual Studio Code, Git
2. Đăng ký và kích hoạt GitHub Student Developer Pack cùng GitHub Copilot
3. Sử dụng được các công cụ AI hỗ trợ lập trình qua dòng lệnh (Copilot CLI, Gemini CLI)
4. Thực hiện thành thạo quy trình Git cơ bản: clone, commit, branch, merge, push
5. Khởi tạo và chạy được project Next.JS đầu tiên

Yêu cầu đầu vào

Kiến thức nền:

- Biết sử dụng máy tính và terminal/command line cơ bản
- Có kiến thức cơ bản về HTML, CSS, JavaScript
- Hiểu biết sơ lược về hệ thống quản lý phiên bản (version control)

Phần cứng:

- Máy tính cá nhân (laptop) có RAM ≥ 8GB, ổ cứng trống ≥ 20GB
- Kết nối Internet ổn định

Tài khoản cần chuẩn bị trước:

- Tài khoản GitHub (đăng ký tại github.com (https://github.com))
- Email sinh viên (.edu) hoặc giấy xác nhận sinh viên (để đăng ký Student Developer Pack)

Quy ước ký hiệu

Ký hiệu	Ý nghĩa
💡	Mẹo hữu ích
⚠️	Lưu ý quan trọng

Phần 1: Cài đặt môi trường phát triển

Thời lượng: *Thời lượng: ~40 phút*

Trong phần này, chúng ta sẽ cài đặt 3 công cụ nền tảng cần thiết cho toàn bộ môn học: **Node.js** (runtime cho JavaScript), **Visual Studio Code** (trình soạn thảo code), và **Git** (hệ thống quản lý phiên bản).

1.1. Cài đặt Node.js

Node.js là môi trường chạy JavaScript phía server, đồng thời cung cấp `npm` (Node Package Manager) — công cụ quản lý thư viện mà chúng ta sẽ sử dụng xuyên suốt môn học.

Cách 1: Cài đặt trực tiếp (đơn giản)

Truy cập https://nodejs.org (https://nodejs.org) và tải phiên bản **LTS** (Long Term Support).

Windows:

- Tải file `.msi` từ trang chủ
- Chạy file cài đặt, nhấn **Next** theo các bước mặc định
- Đảm bảo tích chọn "Add to PATH"

macOS:

- Tải file `.pkg` từ trang chủ
- Chạy file cài đặt theo hướng dẫn

Hoặc cài qua Homebrew:

```bash
brew install node
```

Linux (Ubuntu/Debian):

```bash
sudo apt update
sudo apt install nodejs npm
```

Cách 2: Cài đặt qua nvm (khuyến nghị)

`nvm` (Node Version Manager) cho phép cài đặt và quản lý nhiều phiên bản Node.js trên cùng một máy. Đây là cách được khuyến nghị vì giúp dễ dàng chuyển đổi phiên bản khi cần.

macOS / Linux:

```bash
# Cài đặt nvm
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh | bash

# Khởi động lại terminal, sau đó cài Node.js LTS
nvm install --lts

# Kiểm tra phiên bản đã cài
nvm list
```

Windows:

Tải và cài đặt `nvm-windows` từ [https://github.com/coreybutler/nvm-windows/releases (https://github.com/coreybutler/nvm-windows/releases)](https://github.com/coreybutler/nvm-windows/releases):

- Tải file `nvm-setup.exe`
- Chạy file cài đặt theo hướng dẫn
- Mở **Command Prompt** hoặc **PowerShell** (với quyền Administrator):

```bash
nvm install lts
nvm use lts
```

Kiểm tra cài đặt

Mở terminal (Terminal trên macOS/Linux, Command Prompt hoặc PowerShell trên Windows) và chạy:

```bash
node -v
```

Kết quả mong đợi: Kết quả mong đợi: hiển thị phiên bản Node.js, ví dụ `v22.x.x`

```bash
npm -v
```

Kết quả mong đợi: Kết quả mong đợi: hiển thị phiên bản npm, ví dụ `10.x.x`

Mẹo: `npm` được cài tự động cùng Node.js. Đây là công cụ quản lý thư viện (package) mà chúng ta sẽ dùng rất nhiều.

1.2. Cài đặt Visual Studio Code

Visual Studio Code (VS Code) là trình soạn thảo code miễn phí, nhẹ nhưng mạnh mẽ, được sử dụng rộng rãi nhất trong ngành phát triển phần mềm hiện nay.

Cài đặt

Truy cập https://code.visualstudio.com (https://code.visualstudio.com) và tải phiên bản phù hợp với hệ điều hành.

Windows: Tải file `.exe`, chạy cài đặt. Đảm bảo tích chọn:

- **Kết quả mong đợi:** "Add to PATH"
- **Kết quả mong đợi:** "Register Code as an editor for supported file types"
- **Kết quả mong đợi:** "Add Open with Code action to Windows Explorer"

macOS: Tải file `.zip`, giải nén và kéo vào thư mục **Applications**.

Linux (Ubuntu/Debian):

```bash
sudo apt update
sudo apt install code
```

Hoặc tải file `.deb` từ trang chủ và cài đặt:

```bash
sudo dpkg -i code_*.deb
```

Kiểm tra cài đặt

Mở terminal và chạy:

```bash
code --version
```

Kết quả mong đợi: Kết quả mong đợi: hiển thị phiên bản VS Code

Cài đặt Extension cần thiết

Mở VS Code, nhấn `Ctrl+Shift+X` (Windows/Linux) hoặc `Cmd+Shift+X` (macOS) để mở **Extensions Marketplace**. Tìm và cài đặt các extension sau:

Extension	Mô tả
ESLint	Kiểm tra lỗi và style code JavaScript/TypeScript
Prettier - Code formatter	Tự động format code
Tailwind CSS IntelliSense	Gợi ý class Tailwind CSS
ES7+ React/Redux/React-Native snippets	Snippet code nhanh cho React
Auto Rename Tag	Tự động đổi tên thẻ HTML đóng/mở
Material Icon Theme	Icon đẹp cho file/folder

Cấu hình VS Code cơ bản

Mở **Settings** bằng `Ctrl+,` (Windows/Linux) hoặc `Cmd+,` (macOS), tìm và bật các cài đặt sau:

- **Format On Save:** Tìm "Format On Save" → Tích chọn Kết quả mong đợi:
- **Default Formatter:** Tìm "Default Formatter" → Chọn "Prettier - Code formatter"
- **Tab Size:** Tìm "Tab Size" → Đặt giá trị `2`

Hoặc mở file `settings.json` (nhấn `Ctrl+Shift+P` → gõ "Open User Settings JSON") và thêm:

```Json
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.tabSize": 2,
  "editor.wordWrap": "on",
  "files.autoSave": "afterDelay"
}
```

1.3. Cài đặt Git

Git là hệ thống quản lý phiên bản phân tán, cho phép theo dõi lịch sử thay đổi của code và cộng tác làm việc nhóm hiệu quả.

Cài đặt

Windows:

Tải từ https://git-scm.com (https://git-scm.com) và chạy file cài đặt. Trong quá trình cài đặt:

- Chọn **VS Code** làm default editor
- Chọn "Git from the command line and also from 3rd-party software"
- Các tùy chọn còn lại giữ mặc định

macOS:

```bash
# Cách 1: Cài qua Xcode Command Line Tools (thường đã có sẵn)
xcode-select --install

# Cách 2: Cài qua Homebrew
brew install git
```

Linux (Ubuntu/Debian):

```bash
sudo apt update
sudo apt install git
```

Kiểm tra cài đặt

```bash
git --version
```

Kết quả mong đợi: Kết quả mong đợi: `git version 2.x.x`

Cấu hình ban đầu

Sau khi cài đặt, cấu hình thông tin cá nhân (thay bằng tên và email thật của bạn):

```bash
git config --global user.name "Nguyen Van A"
git config --global user.email "nguyenvana@sv.dlu.edu.vn"
```

Cấu hình thêm một số tùy chọn hữu ích:

```bash
# Đặt default branch là main (thay vì master)
git config --global init.defaultBranch main

# Đặt VS Code làm editor mặc định cho Git
```

```
git config --global core.editor "code --wait"
```

Kiểm tra cấu hình:

```bash
git config --global --list
```

Tạo SSH Key cho GitHub

SSH Key cho phép kết nối đến GitHub mà không cần nhập mật khẩu mỗi lần push/pull.

Bước 1: Tạo SSH Key

```bash
ssh-keygen -t ed25519 -C "nguyenvana@sv.dlu.edu.vn"
```

- Khi được hỏi "Enter file in which to save the key", nhấn **Enter** để dùng đường dẫn mặc định
- Nhập passphrase (hoặc để trống nếu muốn, nhấn **Enter**)

Bước 2: Khởi động SSH Agent và thêm key

```bash
# macOS / Linux
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

```bash
# Windows (PowerShell)
Get-Service -Name ssh-agent | Set-Service -StartupType Manual
Start-Service ssh-agent
ssh-add $env:USERPROFILE\.ssh\id_ed25519
```

Bước 3: Sao chép public key

```bash
# macOS
cat ~/.ssh/id_ed25519.pub | pbcopy

# Linux
cat ~/.ssh/id_ed25519.pub | xclip -selection clipboard

# Windows (PowerShell)
Get-Content $env:USERPROFILE\.ssh\id_ed25519.pub | Set-Clipboard
```

Hoặc đơn giản hơn, hiển thị key và copy thủ công:

```bash
cat ~/.ssh/id_ed25519.pub
```

Bước 4: Thêm SSH Key vào GitHub

1. Truy cập https://github.com/settings/keys (https://github.com/settings/keys)
2. Nhấn **New SSH Key**
3. Đặt tiêu đề (ví dụ: "Laptop cá nhân")
4. Dán public key vào ô **Key**
5. Nhấn **Add SSH Key**

Bước 5: Kiểm tra kết nối

```bash
ssh -T git@github.com
```

Kết quả mong đợi: Kết quả mong đợi: `Hi username! You've successfully authenticated, but GitHub does not provide shell access.`

Bài tập tự làm — Phần 1

1. Cài đặt thành công **Node.js**, **VS Code**, và **Git** trên máy tính cá nhân
2. Chụp ảnh màn hình kết quả các lệnh `node -v`, `npm -v`, `git --version`
3. Cài đặt tất cả Extension đã liệt kê trong VS Code
4. Cấu hình Format On Save và Default Formatter
5. Tạo SSH Key và thêm vào tài khoản GitHub
6. Chạy lệnh `ssh -T git@github.com` và chụp ảnh kết quả thành công

Phần 2: GitHub Student Developer Pack & GitHub Copilot

Thời lượng: *Thời lượng: ~30 phút*

GitHub Student Developer Pack là gói ưu đãi dành cho sinh viên, bao gồm nhiều công cụ và dịch vụ miễn phí có giá trị cao. Một trong những quyền lợi nổi bật nhất là **GitHub Copilot** — trợ lý AI hỗ trợ viết code trực tiếp trong VS Code.

2.1. Đăng ký GitHub Student Developer Pack

Bước 1: Truy cập https://education.github.com/pack (https://education.github.com/pack)

Bước 2: Nhấn **Sign up for Student Developer Pack** (hoặc **Get your pack**)

Bước 3: Đăng nhập bằng tài khoản GitHub

Bước 4: Xác minh trạng thái sinh viên bằng một trong các cách:

- **Email .edu:** Nếu trường cấp email sinh viên có đuôi `.edu` hoặc `.edu.vn`, sử dụng email này
- **Tải lên giấy tờ:** Chụp ảnh thẻ sinh viên hoặc giấy xác nhận sinh viên còn hiệu lực

Bước 5: Điền thông tin trường học:

- Tên trường: `Dalat University` hoặc `Đại học Đà Lạt`
- Mục đích sử dụng: ghi ngắn gọn (ví dụ: "Learning modern web development technologies")

Bước 6: Nhấn **Submit** và chờ xét duyệt

Lưu ý: Thời gian xét duyệt có thể từ vài phút đến vài ngày. Trong thời gian chờ, sinh viên vẫn có thể tiếp tục các phần khác của bài thực hành.

Mẹo: Một số quyền lợi nổi bật của Student Developer Pack:

- **GitHub Copilot** miễn phí (trị giá $10/tháng)
- **GitHub Pro** miễn phí
- Tên miền `.me` miễn phí từ Namecheap
- Credit miễn phí từ nhiều dịch vụ cloud (DigitalOcean, Azure, v.v.)
- Và nhiều công cụ khác

2.2. Kích hoạt GitHub Copilot

Sau khi Student Developer Pack được duyệt:

Bước 1: Truy cập https://github.com/settings/copilot (https://github.com/settings/copilot)

Bước 2: Kích hoạt GitHub Copilot (miễn phí cho sinh viên)

Bước 3: Cài đặt extension trong VS Code:

1. Mở VS Code
2. Nhấn `Ctrl+Shift+X` (Windows/Linux) hoặc `Cmd+Shift+X` (macOS)
3. Tìm **"GitHub Copilot"**
4. Nhấn **Install**
5. Đăng nhập tài khoản GitHub khi được yêu cầu

Kết quả mong đợi: Sau khi cài đặt, biểu tượng Copilot sẽ xuất hiện ở thanh trạng thái (status bar) phía dưới VS Code.

2.3. Trải nghiệm GitHub Copilot trong VS Code

Autocomplete (Gợi ý code tự động)

1. Tạo file mới `demo.js` trong VS Code
2. Gõ comment mô tả chức năng muốn viết:

```javascript
// Hàm kiểm tra một số có phải là số nguyên tố hay không
```

1. Nhấn **Enter** và chờ Copilot gợi ý code
2. Nhấn **Tab** để chấp nhận gợi ý, hoặc **Esc** để bỏ qua

Mẹo: Copilot sẽ tự động đề xuất phần thân hàm dựa trên comment. Bạn có thể nhấn `Alt+]` để xem gợi ý tiếp theo, `Alt+[` để xem gợi ý trước đó.

Copilot Chat (Trò chuyện với AI)

1. Nhấn `Ctrl+Shift+I` (Windows/Linux) hoặc `Cmd+Shift+I` (macOS) để mở **Copilot Chat**
2. Hoặc nhấn vào biểu tượng Copilot ở thanh bên trái
3. Thử hỏi:

```
Giải thích đoạn code kiểm tra số nguyên tố ở trên
```

```
Viết hàm tính giai thừa bằng đệ quy trong JavaScript
```

Inline Chat (Chat trong code)

1. Đặt con trỏ vào dòng code bất kỳ
2. Nhấn `Ctrl+I` (Windows/Linux) hoặc `Cmd+I` (macOS)
3. Gõ yêu cầu, ví dụ: "Thêm xử lý lỗi cho hàm này"
4. Copilot sẽ đề xuất thay đổi ngay trong code

Giải thích: GitHub Copilot sử dụng AI để phân tích ngữ cảnh code và đưa ra gợi ý phù hợp. Tuy nhiên, **luôn kiểm tra kỹ code do AI tạo ra** trước khi sử dụng — AI có thể đưa ra code không chính xác hoặc không tối ưu.

Bài tập tự làm — Phần 2

1. Hoàn thành đăng ký GitHub Student Developer Pack
2. Cài đặt extension GitHub Copilot trong VS Code
3. Tạo file `bai-tap-copilot.js` và sử dụng Copilot để viết:
   - Hàm kiểm tra số nguyên tố
   - Hàm đảo ngược chuỗi
   - Hàm tìm số lớn nhất trong mảng
4. Sử dụng Copilot Chat để giải thích từng hàm đã viết
5. Thử dùng Inline Chat để yêu cầu Copilot thêm comment tiếng Việt vào code

Phần 3: Trải nghiệm Copilot CLI & Gemini CLI

Thời lượng: *Thời lượng: ~30 phút*

Ngoài việc hỗ trợ viết code trong VS Code, các công cụ AI còn có thể giúp bạn trực tiếp trên dòng lệnh (terminal). Trong phần này, chúng ta sẽ trải nghiệm 2 công cụ: **GitHub Copilot CLI** (trợ lý AI trên terminal từ GitHub) và **Gemini CLI** (của Google).

3.1. GitHub Copilot CLI

GitHub Copilot CLI là trợ lý AI chạy trực tiếp trên terminal, được phát triển bởi GitHub. Đây là một công cụ **standalone** (độc lập), cho phép bạn tương tác với AI ngay trong terminal để viết code, lập kế hoạch, giải thích lệnh, và thực hiện các tác vụ phát triển phần mềm một cách tự động.

Lưu ý phân biệt: GitHub Copilot CLI (`@github/copilot`) là sản phẩm **khác hoàn toàn** so với lệnh `gh copilot` (extension của GitHub CLI). Copilot CLI là một agent AI đầy đủ tính năng, trong khi `gh copilot` chỉ cung cấp 2 lệnh đơn giản là `suggest` và `explain`.

Trang chính thức: https://github.com/features/copilot/cli (https://github.com/features/copilot/cli)

Cài đặt Copilot CLI

Cách 1: Cài qua npm (khuyến nghị — đã cài Node.js ở Phần 1)

```bash
npm install -g @github/copilot
```

Cách 2: Cài nhanh qua script

```bash
curl -fsSL https://gh.io/copilot-install | bash
```

Cách 3: Cài qua Homebrew (macOS)

```bash
brew install github-copilot
```

Kiểm tra cài đặt:

```bash
copilot --version
```

Kết quả mong đợi: hiển thị phiên bản Copilot CLI

Xác thực

Khi chạy lần đầu, Copilot CLI sẽ yêu cầu đăng nhập tài khoản GitHub qua trình duyệt:

```bash
copilot
```

Làm theo hướng dẫn trên màn hình để xác thực. Sau khi đăng nhập thành công, bạn sẽ vào giao diện chat tương tác.

Mẹo: Copilot CLI sử dụng chung tài khoản GitHub Copilot — nếu bạn đã kích hoạt qua Student Developer Pack ở Phần 2, bạn đã có quyền truy cập.

Sử dụng cơ bản

Sau khi khởi động, Copilot CLI hiển thị giao diện chat tương tác. Bạn có thể gõ câu hỏi hoặc yêu cầu trực tiếp:

Hỏi đáp về code:

```
> Viết hàm kiểm tra số nguyên tố trong JavaScript
```

Yêu cầu giải thích lệnh:

```
> Giải thích lệnh: find . -name '*.js' -type f
```

Gợi ý lệnh terminal:

```
> Cho tôi lệnh để đếm số dòng code trong tất cả file .js của thư mục hiện tại
```

Tạo file và code:

```
> Tạo file hello.js với nội dung in ra "Xin chào từ CTK46!"
```

Copilot CLI có thể **đọc và chỉnh sửa file** trực tiếp trên máy bạn. Mỗi thao tác sẽ yêu cầu xác nhận trước khi thực thi.

Các tính năng nổi bật

Lệnh / Tính năng	Mô tả
`/plan`	Lập kế hoạch chi tiết trước khi code (rất hữu ích cho bài tập lớn)
`/model`	Chuyển đổi giữa các mô hình AI (Claude, GPT, Gemini)
`/fleet`	Chạy nhiều agent song song để tăng tốc
`/help`	Xem danh sách đầy đủ các lệnh
MCP	Tích hợp với GitHub: issues, pull requests, branches

Mẹo: Copilot CLI rất hữu ích khi bạn muốn viết code, lập kế hoạch, hoặc thực hiện các tác vụ phức tạp ngay trên terminal — không cần mở trình duyệt hay editor.

3.2. Gemini CLI

Gemini CLI là công cụ agentic AI từ Google, mang sức mạnh của mô hình Gemini trực tiếp vào terminal. Tương tự như GitHub Copilot CLI, Gemini CLI là một **agent AI đầy đủ tính năng** — có thể đọc/sửa file, chạy lệnh shell, quản lý codebase lớn, tạo ứng dụng từ hình ảnh hoặc PDF, và tự động hóa các workflow phức tạp.

Trang chính thức: https://geminicli.com (https://geminicli.com) | GitHub: https://github.com/google-gemini/gemini-cli (https://github.com/google-gemini/gemini-cli)

Cài đặt Gemini CLI

Yêu cầu hệ thống: Node.js phiên bản 20.0.0 trở lên (đã cài ở Phần 1).

Cách 1: Cài qua npm (khuyến nghị)

```bash
npm install -g @google/gemini-cli
```

Cách 2: Cài qua Homebrew (macOS/Linux)

```bash
brew install gemini-cli
```

Cách 3: Chạy trực tiếp mà không cần cài đặt (dùng npx)

```bash
npx @google/gemini-cli
```

Kiểm tra cài đặt:

```bash
gemini --version
```

Kết quả mong đợi: Kết quả mong đợi: hiển thị phiên bản Gemini CLI

Lưu ý: Hãy kiểm tra trang chính thức tại https://geminicli.com/docs/get-started/installation (https://geminicli.com/docs/get-started/installation) để xem hướng dẫn cài đặt mới nhất.

Xác thực

Gemini CLI sử dụng đăng nhập Google hoặc API key. Khi chạy lần đầu, CLI sẽ hướng dẫn bạn qua quá trình cấu hình.

Cách 1: Đăng nhập bằng tài khoản Google (đơn giản nhất — khuyến nghị)

```bash
gemini
```

Khi được hỏi "How would you like to authenticate for this project?", chọn **Sign in with Google**. Trình duyệt sẽ mở ra để bạn đăng nhập với tài khoản Google. Sau khi đăng nhập thành công, thông tin xác thực được lưu cục bộ cho các phiên sau.

Cách 2: Sử dụng API Key

1. Truy cập https://aistudio.google.com/apikey (https://aistudio.google.com/apikey)
2. Tạo API key mới
3. Đặt biến môi trường:

```bash
# macOS / Linux
export GEMINI_API_KEY="YOUR_API_KEY"

# Windows (PowerShell)
$env:GEMINI_API_KEY="YOUR_API_KEY"
```

1. Khởi động CLI và chọn **Use Gemini API key**:

```bash
gemini
```

Lưu ý: Với API key miễn phí, giới hạn là 250 requests/ngày và chỉ dùng được model Flash.

Mẹo: Sinh viên nên sử dụng **Cách 1** (đăng nhập Google) vì có quota cao hơn và hỗ trợ nhiều model hơn.

Sử dụng Gemini CLI

Chế độ tương tác (REPL) — cách sử dụng chính:

Khởi động Gemini CLI và tương tác trực tiếp trong giao diện chat:

```bash
gemini
```

Sau khi vào giao diện tương tác, gõ câu hỏi hoặc yêu cầu trực tiếp:

```
> Viết hàm kiểm tra số nguyên tố trong JavaScript
```

Tham chiếu file trực tiếp với `@`:

Dùng ký hiệu `@` để đưa nội dung file vào ngữ cảnh:

```
> @src/app.js Giải thích file này làm gì
```

```
> @src/components/Login.tsx @src/types/User.ts Refactor component để sử dụng
interface User mới
```

Đọc và chỉnh sửa file:

Gemini CLI có thể đọc, sửa, và tạo file trực tiếp trên máy. Mỗi thao tác sẽ hiển thị diff và yêu cầu xác nhận trước khi thực thi:

```
> Tạo file hello.js với nội dung in ra "Xin chào từ CTK46!"
```

```
> Thêm xử lý lỗi cho hàm fetchData trong @src/api.js
```

Chạy lệnh shell:

```
> Chạy test cho project này
```

Chế độ non-interactive (chạy 1 câu lệnh rồi thoát):

```bash
gemini -p "Giải thích lỗi TypeError: Cannot read properties of undefined"
```

Các tính năng nổi bật

Lệnh / Tính năng	Mô tả
`gemini`	Khởi động chế độ tương tác (REPL)
`@file`	Tham chiếu file trực tiếp trong prompt
`--model pro/flash/flash-lite`	Chọn model: pro (mạnh), flash (cân bằng), flash-lite (nhanh)
`--sandbox`	Chạy trong môi trường sandbox an toàn
`--approval-mode=plan`	Chế độ lập kế hoạch (chỉ đọc, không sửa file)
`--resume latest`	Tiếp tục phiên làm việc gần nhất
`/help`	Xem danh sách đầy đủ các lệnh
`/quit`	Thoát phiên tương tác
`GEMINI.md`	File ngữ cảnh project (tương tự COPILOT.md)
`.geminiignore`	File loại trừ — ẩn file/thư mục khỏi AI
Extensions	Mở rộng tính năng bằng extensions: `gemini extensions install <url>`
MCP Server	Kết nối với MCP server: `gemini mcp add <name> <command>`
Agent Skills	Kỹ năng chuyên biệt cho các tác vụ cụ thể

Mẹo: Gemini CLI là công cụ agentic AI mạnh mẽ — không chỉ hỏi đáp mà còn có thể đọc/sửa file, chạy lệnh, quản lý project, tương tự như GitHub Copilot CLI.

3.3. So sánh Copilot CLI và Gemini CLI

Tiêu chí	GitHub Copilot CLI	Gemini CLI
Nhà phát triển	GitHub (Microsoft)	Google
Cài đặt	`npm install -g @github/copilot`	`npm install -g @google/gemini-cli`
Tính chất	Agent AI tự động trên terminal — đọc/sửa file, chạy lệnh, tạo PR	Agent AI tự động trên terminal — đọc/sửa file, chạy lệnh, quản lý codebase
Tích hợp	Tích hợp sâu với GitHub (issues, PRs, branches) qua MCP	Hỗ trợ MCP server, extensions, agent skills, Google Cloud
Tính năng nổi bật	`/plan`, `/fleet`, `/model`, `/delegate`, multi-agent	Plan mode, sandbox, extensions, MCP, skills, `@` file reference

Đa mô hình	Hỗ trợ Claude, GPT, Gemini — chuyển đổi bằng `/model`	Hỗ trợ Gemini Pro, Flash, Flash-Lite — chuyển đổi bằng `--model`
Chi phí	Miễn phí cho sinh viên (Student Pack)	Miễn phí 1.000 requests/ngày (tài khoản Google cá nhân)
Phù hợp cho	Viết code, lập kế hoạch, tự động hóa workflow phát triển	Viết code, quản lý codebase, tự động hóa workflow, tạo app

Giải thích: Cả hai công cụ đều là **agent AI đầy đủ tính năng** trên terminal. Không có công cụ nào "tốt nhất" — mỗi công cụ có hệ sinh thái riêng. Lập trình viên giỏi biết sử dụng đúng công cụ cho đúng tác vụ.

Bài tập tự làm — Phần 3

1. Cài đặt GitHub Copilot CLI (`npm install -g @github/copilot`) và đăng nhập thành công
2. Sử dụng Copilot CLI để thực hiện 3 tác vụ sau:
   - Yêu cầu Copilot viết hàm tính tổng các số chẵn trong mảng (JavaScript)
   - Yêu cầu Copilot giải thích lệnh `find . -name '*.js' -type f -exec wc -l {} +`
   - Yêu cầu Copilot tạo file `hello.js` với nội dung in ra thông tin cá nhân của bạn
3. Thử sử dụng lệnh `/plan` để lập kế hoạch cho 1 tác vụ đơn giản (ví dụ: tạo trang web giới thiệu bản thân)
4. Cài đặt Gemini CLI (`npm install -g @google/gemini-cli`) và đăng nhập bằng tài khoản Google
5. Sử dụng Gemini CLI ở chế độ tương tác (`gemini`) để thực hiện 3 tác vụ sau:
   - Dùng `@` để tham chiếu 1 file code và yêu cầu Gemini giải thích
   - Yêu cầu Gemini tạo 1 file mới (ví dụ: `hello.js` với nội dung in ra thông tin cá nhân)
   - Thử chế độ lập kế hoạch: `gemini --approval-mode=plan` và hỏi "Lập kế hoạch tạo trang web giới thiệu bản thân"

6. Ghi lại nhận xét cá nhân: bạn thấy công cụ nào hữu ích hơn và tại sao?

Phần 4: Thực hành Git workflow cơ bản

Thời lượng: *Thời lượng: ~40 phút*

Git là công cụ không thể thiếu trong phát triển phần mềm hiện đại. Phần này hướng dẫn quy trình Git cơ bản mà bạn sẽ sử dụng xuyên suốt môn học và trong công việc thực tế.

4.1. Tạo repository trên GitHub

1. Truy cập https://github.com/new (https://github.com/new)
2. Điền thông tin:
   - **Repository name:** `ctk46-lab01` (hoặc tên bạn chọn)
   - **Description:** "Bài thực hành 1 - Thiết lập môi trường phát triển"
   - **Visibility:** Public
   - **Kết quả mong đợi:** Tích chọn **Add a README file**
   - **Add .gitignore:** Chọn **Node**
3. Nhấn **Create repository**

4.2. Clone repository về máy

```bash
# Clone bằng SSH (khuyến nghị - đã cấu hình SSH Key ở Phần 1)
git clone git@github.com:TEN_TAI_KHOAN/ctk46-lab01.git

# Hoặc clone bằng HTTPS
git clone https://github.com/TEN_TAI_KHOAN/ctk46-lab01.git
```

Di chuyển vào thư mục project:

```bash
cd ctk46-lab01
```

Kiểm tra trạng thái:

```bash
git status
```

Kết quả mong đợi: Kết quả: `On branch main, nothing to commit, working tree clean`

4.3. Quy trình làm việc cơ bản

Quy trình Git cơ bản gồm 4 bước: **Thay đổi → Stage → Commit → Push**

Bước 1: Tạo hoặc chỉnh sửa file

```bash
# Tạo file mới
echo "console.log('Xin chào từ CTK46!');" > hello.js
```

Bước 2: Kiểm tra trạng thái thay đổi

```bash
git status
```

Kết quả mong đợi: Kết quả: file `hello.js` xuất hiện trong mục "Untracked files" (màu đỏ)

Bước 3: Thêm file vào staging area

```bash
# Thêm 1 file cụ thể
git add hello.js

# Hoặc thêm tất cả file đã thay đổi
git add .
```

```bash
git status
```

Kết quả mong đợi: Kết quả: file `hello.js` xuất hiện trong mục "Changes to be committed" (màu xanh)

Bước 4: Commit (lưu thay đổi vào lịch sử)

```bash
git commit -m "feat: thêm file hello.js với lời chào"
```

Bước 5: Push lên GitHub

```bash
git push origin main
```

Kết quả mong đợi: Kiểm tra trên GitHub: file `hello.js` đã xuất hiện trong repository

Giải thích: Giải thích luồng dữ liệu:

```
Working Directory → (git add) → Staging Area → (git commit) → Local Repo → (git
push) → Remote Repo (GitHub)
```

4.4. Conventional Commits

Conventional Commits là quy ước viết commit message theo format chuẩn, giúp lịch sử commit dễ đọc và có thể tự động hóa (changelog, versioning).

Format:

```
type(scope): message
```

Các type phổ biến:

Type	Ý nghĩa	Ví dụ
`feat`	Thêm tính năng mới	`feat: thêm trang đăng nhập`
`fix`	Sửa lỗi	`fix: sửa lỗi hiển thị trên mobile`
`docs`	Thay đổi tài liệu	`docs: cập nhật README`
`style`	Thay đổi format code (không ảnh hưởng logic)	`style: format lại code theo prettier`
`refactor`	Tái cấu trúc code (không thêm tính năng, không sửa lỗi)	`refactor: tách hàm xử lý dữ liệu`
`chore`	Thay đổi khác (config, build, v.v.)	`chore: cập nhật dependencies`
`test`	Thêm hoặc sửa test	`test: thêm unit test cho hàm login`

Ví dụ thực hành:

```bash
# Chỉnh sửa README
echo "# Bài Thực Hành 1" > README.md
echo "" >> README.md
echo "Đây là project thực hành cho môn Các công nghệ mới trong PTPM." >> README.md
git add README.md
```

```
git commit -m "docs: cập nhật nội dung README"

# Thêm file mới
echo "function add(a, b) { return a + b; }" > utils.js
git add utils.js
git commit -m "feat: thêm hàm tiện ích cộng hai số"

# Sửa lỗi
echo "function add(a, b) { return Number(a) + Number(b); }" > utils.js
git add utils.js
git commit -m "fix: xử lý trường hợp tham số không phải số"
```

4.5. Branching (phân nhánh)

Branch cho phép phát triển tính năng mới mà không ảnh hưởng đến code chính (main).

Tạo branch mới:

```bash
# Tạo và chuyển sang branch mới
git checkout -b feature/trang-gioi-thieu
```

Hoặc sử dụng lệnh mới hơn:

```bash
git switch -c feature/trang-gioi-thieu
```

Thực hiện thay đổi trên branch:

```bash
echo "<h1>Giới thiệu</h1>" > about.html
git add about.html
git commit -m "feat: thêm trang giới thiệu"
```

Xem danh sách branch:

```bash
git branch
```

Kết quả mong đợi: Kết quả: hiển thị danh sách branch, branch hiện tại đánh dấu `*`

Chuyển về branch main:

```bash
git checkout main
# hoặc
git switch main
```

Merge branch vào main:

```bash
# Đảm bảo đang ở branch main
git checkout main

# Merge branch feature vào main
git merge feature/trang-gioi-thieu
```

Push tất cả thay đổi:

```bash
git push origin main
```

Xóa branch đã merge (tùy chọn):

```bash
git branch -d feature/trang-gioi-thieu
```

Giải thích: Quy ước đặt tên branch:

- `feature/ten-tinh-nang` — branch phát triển tính năng mới
- `fix/mo-ta-loi` — branch sửa lỗi
- `docs/mo-ta` — branch cập nhật tài liệu

4.6. Giải quyết conflict

Conflict xảy ra khi cùng một dòng code được thay đổi trên 2 branch khác nhau. Git không biết giữ phiên bản nào nên yêu cầu bạn giải quyết thủ công.

Tạo tình huống conflict (để thực hành):

```bash
# Ở branch main, sửa file
git checkout main
echo "const greeting = 'Xin chào!';" > greeting.js
git add greeting.js
git commit -m "feat: thêm biến greeting trên main"

# Tạo branch mới và sửa cùng file
git checkout -b feature/greeting-update
echo "const greeting = 'Hello World!';" > greeting.js
git add greeting.js
git commit -m "feat: cập nhật greeting sang tiếng Anh"

# Quay về main và merge
git checkout main
git merge feature/greeting-update
```

Kết quả mong đợi: Git sẽ báo **CONFLICT**. Mở file `greeting.js` trong VS Code:

```
<<<<<<< HEAD
const greeting = 'Xin chào!';
=======
const greeting = 'Hello World!';
>>>>>>> feature/greeting-update
```

Giải quyết: Chọn phiên bản muốn giữ (hoặc kết hợp cả hai), xóa các dấu `<<<<<<<`, `=======`, `>>>>>>>`:

```Javascript
const greeting = 'Xin chào! Hello World!';
```

Sau đó commit:

```Bash
git add greeting.js
git commit -m "fix: giải quyết conflict greeting"
```

Mẹo: VS Code có giao diện hỗ trợ giải quyết conflict rất trực quan — bạn có thể nhấn các nút **Accept Current Change**, **Accept Incoming Change**, hoặc **Accept Both Changes** ngay trên code.

4.7. File .gitignore

`.gitignore` là file đặc biệt cho Git biết những file/thư mục nào **không** cần theo dõi (ví dụ: `node_modules/`, file cấu hình cá nhân, file build).

Khi tạo repository với template **Node**, GitHub đã tạo sẵn `.gitignore` phù hợp. Xem nội dung:

```Bash
cat .gitignore
```

Một số mục phổ biến trong `.gitignore` cho project Node.js:

```
# Dependencies
node_modules/

# Build output
.next/
dist/
```

```
build/

# Environment variables (chứa thông tin nhạy cảm)
.env
.env.local
.env.production

# IDE
.vscode/
.idea/

# OS files
.DS_Store
Thumbs.db
```

Lưu ý: Quan trọng: File `.env` chứa API key, mật khẩu, thông tin nhạy cảm — **tuyệt đối không commit** lên GitHub!

4.8. Xem lịch sử commit

```Bash
# Xem lịch sử commit ngắn gọn
git log --oneline

# Xem lịch sử chi tiết
git log

# Xem lịch sử với biểu đồ branch
git log --oneline --graph --all
```

Bài tập tự làm — Phần 4

1. Tạo repository `ctk46-lab01` trên GitHub (nếu chưa tạo)
2. Clone về máy và thực hiện các thao tác sau:
   - Tạo ít nhất **5 commits** với message đúng chuẩn Conventional Commits
   - Sử dụng ít nhất **3 type khác nhau** (feat, docs, fix, v.v.)
3. Tạo branch `feature/about-page`, thực hiện thay đổi, và merge vào `main`
4. Tạo tình huống conflict và giải quyết thành công
5. Push tất cả thay đổi lên GitHub
6. Chạy `git log --oneline` và chụp ảnh kết quả (phải có ít nhất 5 commits)

Phần 5: Khởi tạo project Next.JS đầu tiên

Thời lượng: *Thời lượng: ~30 phút*

Next.JS là React framework phổ biến nhất hiện nay để xây dựng ứng dụng web hiện đại. Trong phần này, chúng ta sẽ khởi tạo project Next.JS đầu tiên, tìm hiểu cấu trúc thư mục cơ bản, và chạy ứng dụng trên máy local. Chi tiết về Next.JS sẽ được đề cập sâu ở buổi 2 và 3.

5.1. Khởi tạo project

Đảm bảo bạn đang ở trong thư mục repository đã tạo ở Phần 4:

```bash
cd ctk46-lab01
```

Chạy lệnh khởi tạo project Next.JS:

```bash
npx create-next-app@latest .
```

Lưu ý: Dấu `.` ở cuối lệnh có nghĩa là tạo project ngay trong thư mục hiện tại (không tạo thư mục con mới).

Khi được hỏi các tùy chọn, chọn như sau:

```
✓ Would you like to use TypeScript? … Yes
✓ Would you like to use ESLint? … Yes
✓ Would you like to use Tailwind CSS? … Yes
✓ Would you like your code inside a `src/` directory? … Yes
✓ Would you like to use App Router? (recommended) … Yes
✓ Would you like to use Turbopack for next dev? … Yes
✓ Would you like to customize the import alias (@/* by default)? … No
```

Giải thích: Giải thích các tùy chọn:

Tùy chọn	Lý do chọn
TypeScript	Ngôn ngữ mở rộng của JavaScript, thêm kiểu dữ liệu, giúp phát hiện lỗi sớm
ESLint	Công cụ kiểm tra lỗi và style code tự động
Tailwind CSS	Framework CSS hiện đại, viết style trực tiếp trong HTML/JSX
src/ directory	Tổ chức code gọn gàng hơn, tách source code khỏi file config

App Router	Hệ thống routing mới nhất của Next.JS (thay thế Pages Router cũ)
Turbopack	Công cụ build nhanh hơn webpack, cải thiện tốc độ phát triển

Chờ quá trình cài đặt hoàn tất (có thể mất 1-2 phút).

5.2. Cấu trúc thư mục

Sau khi khởi tạo, cấu trúc thư mục project như sau:

```
ctk46-lab01/
├── src/
│   └── app/
│       ├── layout.tsx        ← Layout chính (bao bọc toàn bộ trang)
│       ├── page.tsx          ← Trang chủ (/)
│       ├── globals.css       ← CSS toàn cục
│       └── favicon.ico       ← Icon trang web
├── public/                   ← File tĩnh (ảnh, font, v.v.)
├── node_modules/             ← Thư viện đã cài (không commit)
├── package.json              ← Thông tin project và dependencies
├── tsconfig.json             ← Cấu hình TypeScript
├── tailwind.config.ts        ← Cấu hình Tailwind CSS
├── next.config.ts            ← Cấu hình Next.JS
├── .gitignore                ← Danh sách file Git bỏ qua
└── README.md                 ← Mô tả project
```

Giải thích: Các file/thư mục quan trọng cần nhớ:

- `src/app/page.tsx` — Nội dung trang chủ, đây là file bạn sẽ chỉnh sửa nhiều nhất lúc đầu
- `src/app/layout.tsx` — Layout chính, chứa HTML bao ngoài (head, body)
- `package.json` — Liệt kê dependencies và scripts
- `public/` — Chứa file tĩnh, truy cập qua URL gốc (ví dụ: `/logo.png`)

5.3. Chạy development server

```bash
npm run dev
```

Kết quả mong đợi: Kết quả mong đợi:

```
  ▲ Next.js (turbo)
  - Local:        http://localhost:3000
  - Environments: .env.local
```

```
✓ Starting...
✓ Ready in xxxms
```

Mở trình duyệt và truy cập **http://localhost:3000**

Kết quả mong đợi: Bạn sẽ thấy trang mặc định của Next.JS với logo và các liên kết hướng dẫn.

Mẹo: Development server có tính năng **Hot Reload** — khi bạn thay đổi code và lưu file, trang web sẽ tự động cập nhật mà không cần refresh trình duyệt.

Để dừng server, nhấn `Ctrl+C` trong terminal.

5.4. Chỉnh sửa trang chủ

Mở file `src/app/page.tsx` trong VS Code. Xóa toàn bộ nội dung mặc định và thay bằng:

```tsx
export default function Home() {
  return (
    <main className="flex min-h-screen flex-col items-center justify-center p-8">
      <div className="text-center">
        <h1 className="text-4xl font-bold mb-4">
          Xin chào! 👋
        </h1>
        <p className="text-xl text-gray-600 mb-2">
          Họ và tên: <strong>Nguyễn Văn A</strong>
        </p>
        <p className="text-xl text-gray-600 mb-2">
          MSSV: <strong>2112345</strong>
        </p>
        <p className="text-xl text-gray-600 mb-6">
          Môn: Các công nghệ mới trong PTPM
        </p>
        <div className="bg-blue-100 rounded-lg p-4">
          <p className="text-blue-800">
            🚀 Đây là project Next.JS đầu tiên của tôi!
          </p>
        </div>
      </div>
    </main>
  );
}
```

Lưu file (`Ctrl+S` hoặc `Cmd+S`). Kiểm tra trình duyệt — trang web tự động cập nhật.

Giải thích: Giải thích nhanh:

- `export default function Home()` — Khai báo component React cho trang chủ
- `className="..."` — Thuộc tính CSS class (dùng `className` thay vì `class` trong JSX)
- Các class như `text-4xl`, `font-bold`, `mb-4` — class của Tailwind CSS (sẽ học chi tiết ở buổi 2)

Thử thêm: Chỉnh sửa tên, MSSV, thêm thông tin cá nhân, thay đổi màu sắc, v.v.

5.5. Chỉnh sửa layout

Mở file `src/app/layout.tsx` và cập nhật thông tin metadata:

```tsx
import type { Metadata } from "next";
import "./globals.css";

export const metadata: Metadata = {
  title: "CTK46 - Lab 01",
  description: "Bài thực hành 1 - Các công nghệ mới trong PTPM",
};

export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  return (
    <html lang="vi">
      <body>{children}</body>
    </html>
  );
}
```

Kết quả mong đợi: Kiểm tra tab trình duyệt — tiêu đề trang đã thay đổi thành "CTK46 - Lab 01".

5.6. Commit và push lên GitHub

Dừng dev server (`Ctrl+C`), sau đó:

```bash
git add .
git commit -m "feat: khởi tạo project Next.JS với thông tin cá nhân"
git push origin main
```

Kết quả mong đợi: Kiểm tra trên GitHub — project Next.JS đã được đẩy lên repository.

Bài tập tự làm — Phần 5

1. Khởi tạo project Next.JS trong repository `ctk46-lab01`
2. Chỉnh sửa trang chủ (`src/app/page.tsx`) với thông tin cá nhân thật:
   - Họ và tên
   - MSSV
   - Lớp
   - Một câu giới thiệu ngắn về bản thân
3. Cập nhật metadata trong `layout.tsx`
4. Thử tạo thêm 1 đoạn nội dung mới trên trang chủ (ví dụ: danh sách sở thích, mục tiêu học tập)
5. Commit với message chuẩn Conventional Commits và push lên GitHub
6. Chụp ảnh trang web chạy trên `localhost:3000`

Phần 6: Tổng kết & Yêu cầu nộp bài

Thời lượng: *Thời lượng: ~5 phút*

Checklist hoàn thành

Trước khi nộp bài, hãy đảm bảo bạn đã hoàn thành tất cả các mục sau:

☐ Cài đặt thành công Node.js, VS Code, Git

☐ Cấu hình VS Code (extensions, format on save)

☐ Tạo SSH Key và kết nối thành công với GitHub

☐ Đăng ký GitHub Student Developer Pack

☐ Cài đặt và sử dụng GitHub Copilot trong VS Code

☐ Cài đặt GitHub Copilot CLI và trải nghiệm các tính năng cơ bản

☐ Trải nghiệm Gemini CLI

☐ Tạo repository trên GitHub với ít nhất 5 commits đúng chuẩn

☐ Thực hành branching và merge thành công

☐ Khởi tạo project Next.JS và chạy trên localhost

☐ Chỉnh sửa trang chủ với thông tin cá nhân

☐ Push toàn bộ code lên GitHub

Yêu cầu nộp bài

Nội dung nộp:

1. Tập tin tài liệu chứa mô tả, hình ảnh của các bài tập (file PDF)
2. **Link GitHub repository** chứa project Next.JS đã hoàn thành (đính kèm trong tài liệu)
3. Repository phải đáp ứng:
   - Có ít nhất **5 commits** với message đúng chuẩn Conventional Commits
   - Trang chủ (`src/app/page.tsx`) đã được chỉnh sửa với thông tin cá nhân
   - File `README.md` mô tả ngắn gọn về project

Hình thức nộp: Nộp tập tin tài liệu (PDF) lên hệ thống LMS

Chuẩn bị cho buổi sau

Buổi 2 sẽ đi sâu vào **Next.JS — Nền tảng & Routing**. Để chuẩn bị tốt, hãy:

1. Đọc trước tài liệu Next.JS App Router: https://nextjs.org/docs/app (https://nextjs.org/docs/app)
2. Xem qua Tailwind CSS cơ bản: https://tailwindcss.com/docs (https://tailwindcss.com/docs)
3. Ôn lại kiến thức React cơ bản (component, props, state)

Tài liệu tham khảo

- Node.js: https://nodejs.org/docs (https://nodejs.org/docs)
- Visual Studio Code: https://code.visualstudio.com/docs (https://code.visualstudio.com/docs)
- Git: https://git-scm.com/doc (https://git-scm.com/doc)
- GitHub Docs: https://docs.github.com (https://docs.github.com)
- GitHub Student Developer Pack: https://education.github.com/pack (https://education.github.com/pack)

- GitHub Copilot: https://docs.github.com/en/copilot (https://docs.github.com/en/copilot)
- GitHub Copilot CLI: https://github.com/features/copilot/cli (https://github.com/features/copilot/cli)
- GitHub Copilot CLI Best Practices: https://docs.github.com/en/copilot/how-tos/copilot-cli/cli-best-practices (https://docs.github.com/en/copilot/how-tos/copilot-cli/cli-best-practices)
- Next.JS: https://nextjs.org/docs (https://nextjs.org/docs)
- Tailwind CSS: https://tailwindcss.com/docs (https://tailwindcss.com/docs)
- Conventional Commits: https://www.conventionalcommits.org (https://www.conventionalcommits.org)